

1. In which of the following operations do Linked List and Arrays have the same time complexity?

- A. Accessing an element at index i**
- B. Inserting at the end**
- C. Searching for a specific value**
- D. Deleting the first element**

2. Which of the following data structures can be used to implement a stack data structure?

- A. Arrays**
- B. Singly linked list**
- C. Doubly linked list**
- D. All the above**

3. Which operation on a singly linked list has the worst-case time complexity of $O(n)$, with the first node as head?

- A. Insert at the beginning**
- B. Insert at the end**
- C. Delete the first node**
- D. They are all the same**

4. Which of the following array characteristics is the reason why arrays can access an element directly by the index?

- A. Homogeneity**
- B. Contiguous memory**
- C. Static data structure**
- D. None of the above**

5. Which of the following data structures is the most suitable for implementing Undo and Redo operations?

- A. Queue**
- B. Array**
- C. Stack**
- D. Linked list**

6. In an algorithm with time complexity $O(2^n)$, if the input size n increases by 3, how much does the algorithm's work increase?

- A. 6 times**
- B. 8 times**
- C. 16 times**
- D. 3 times**

A user performs 3 actions: action1, action2, action3 where action3 is the top.

Then perform Redo once, then Undo twice.

What is currently on top of the stack?

Answer:

What will be the content of a given singly linked list with the first node as head, last node as last, and containing the following data items:

10 → 20 → 30

After executing the function enjoy(5):

```
public void enjoy ( int value ) {  
    head = new Llnode(head.next.data, head);  
    last.next = new Llnode(last.data, null);  
    last = last.next;  
}
```

Answer:

Singly linked list content:

Given a sorted array

Float [] Heights = {150.0, 160.5, 170.2, 175.4, 180.0, 185.7, 190.3}

Using the Binary Search Algorithm, what is the number of comparisons required to find a key value = 185.7 in the array?

Answer:

You are given an array, its elements:

arr = {27, 21, 25, 29, 22, 28, 24}

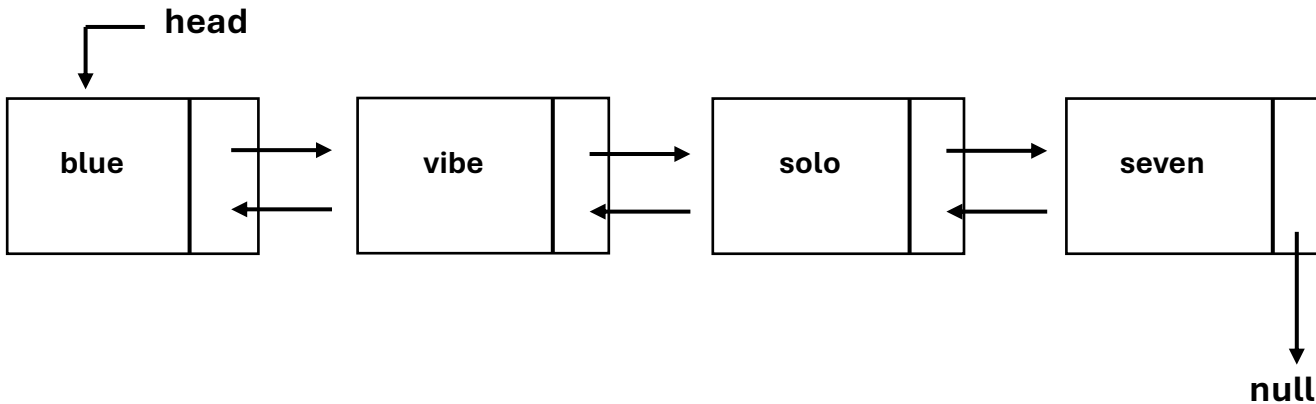
Write the array contents after calling the method adjustArray(arr)

```
public void adjustArray ( int[] arr ) {  
    for ( int i = 1; i < arr.length; i += 2 ) {  
        arr[i] += arr[i - 1] - arr[i + 1];  
    }  
}
```

Answer:

The array content:

Consider the following sorted Doubly Linked List, where head and last are pointing to the first and last nodes in the list respectively, as follows:



Then answer the following questions:

a) What will be the return value after executing the function fun()?

```
public int fun () {  
    int T = 0;  
    DLNode current = head;  
  
    while ( current.next != null ) {  
        T++;  
        current = current.next;  
    }  
    return T;  
}
```

Answer:

b) What will be printed as the result of executing the following code statement?

```
System.out.println(head.next.data);
```

Answer:

c) What will be printed as the result of executing the following code statement?

```
System.out.println(head.next.next.prev.data);
```

Answer:

d) What will be printed as the result of executing the following code statement?

```
current = head.next;
```

```
System.out.println(current.next.next.prev.data);
```

Answer:

e) What will be the content of the Doubly linked list above after executing the following code statements?

```
last = last.prev;
```

```
last.next = null;
```

Answer (list content):

Algorithm runs in $O(2^n)$ time.

**For an input of size 7, the running time is 17 milliseconds.
What will be the problem size if it takes 68 ms to execute?**

Solution:

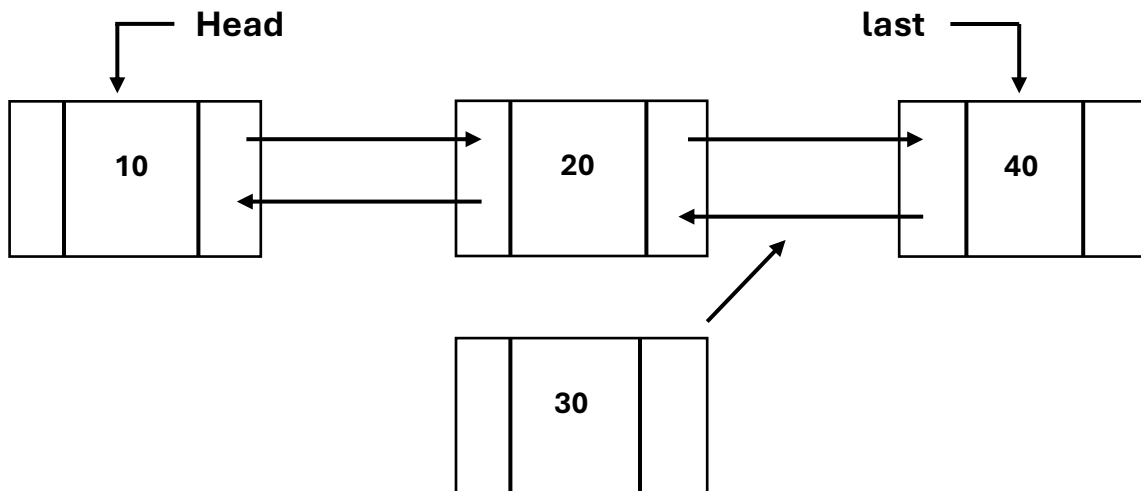
What will be printed given the following code segment for a given singly linked list with the first node as head, helpPtr = head and contains the following data items: 5 → 10 → 15

```
public void enjoy ( LLnode helpPtr ) {  
    if ( helpPtr == null ) return;  
  
    System.out.print(helpPtr.data + " ");  
    enjoy(helpPtr.next);  
    System.out.print(helpPtr.data + " ");  
}
```

Output:

Given a sorted doubly linked list in increasing order, where head and last are pointing to the first and last node.

Insert the node 30 into the list in the correct sorted position (no need to traverse).



a. Insert 30 in correct position:

```
DLLNode newNode =
```

```
newNode.next =
```

```
newNode.prev = last.prev;
```

```
                = newNode;
```

```
                = newNode;
```

b. Write segments to delete the last node and update the last pointer:

Determine the Big-O running time of the following code, you must explain your answer:

```
for (int i = 0; i < n; i++) {  
    for (int j = 1; j <= n; j++) {  
        int k = 1;  
        while (k < n) {  
            k = k * 2;  
        }  
    }  
}
```

Solution: $O(\quad)$

Explanation:

If N is even, add N and continue with N/2

If N is odd, add N and continue with N-1.

The method returns when N = 1.

Example:

If N = 10, the method will return 22.

$$10 + 5 + 4 + 2 + 1 = 22$$

Write the recursive method:

```
public int rec ( int N ) {
```

```
    if (      )
```

```
        return ;
```

```
    if (      )
```

```
        return
```

```
    else
```

```
        return
```

```
}
```

Postfix Expression:

You are given a stack implemented using an array
The stack always have odd number of elements

```
public static void main (String[] args) {  
    Stack s = new Stack ();  
    s.push(1);  
    s.push(2);  
    s.push(3);  
    s.push(4);  
    s.push(5);  
}
```

Task:

Write a method that removes the middle element from the stack.

***Hint:* middle=top/2;**

**You can use another stack, and the operations: pop, push, isEmpty()
ONLY**

The method is in the Stack class

Solution:

```
public void deleteMiddle () {
```

```
}
```